St Atiqoh (6026252006)

Tugas 2 – RASA Customization

Question-1: Requirements analysis and architecture of chatbot *(Soal 1: Analisis kebutuhan dan arsitektur chatbot)*

a. Explain the primary function of the RASA Framework in customer service chatbot development. List the RASA components required to handle message understanding, dialogue management, and action execution. *(a. Jelaskan fungsi utama RASA Framework dalam pengembangan chatbot layanan pelanggan. Sebutkan komponen RASA yang diperlukan untuk menangani pemahaman pesan, pengelolaan dialog, dan eksekusi aksi.)*

Pada tugas ini saya membangun chatbot customer service untuk sebuah online shop fiktif bernama OlshopKu. Chatbot bernama "Mimin" ini berperan sebagai admin toko yang menjawab pertanyaan pembeli dalam Bahasa Indonesia, seperti ketersediaan stok, harga dan diskon, informasi produk, daftar produk per kategori, status pesanan, serta data dan riwayat transaksi pelanggan. Proses pemesanan dan pembayaran tidak ditangani oleh chatbot. Jika pembeli menanyakan hal tersebut, chatbot akan mengarahkannya ke halaman checkout atau ke admin.

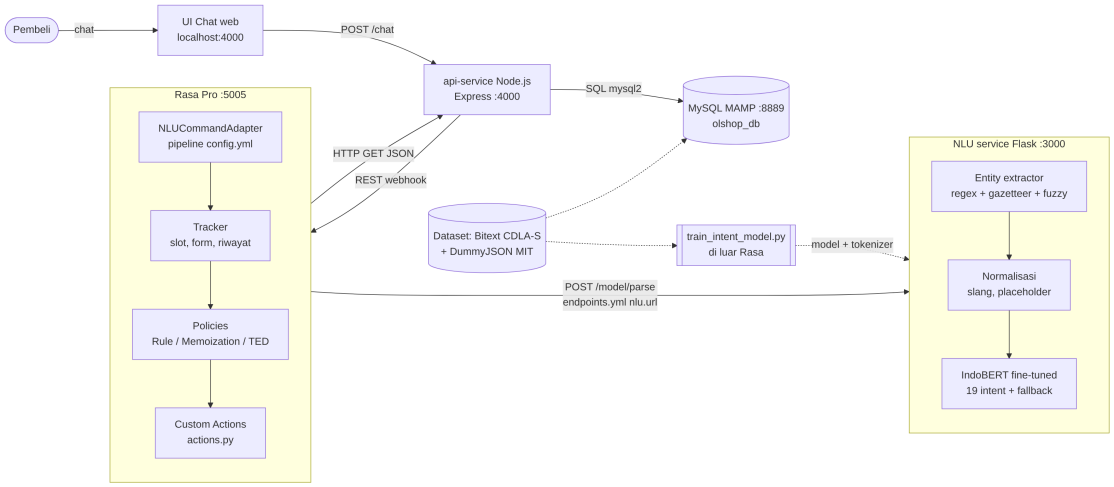
Rasa merupakan framework open-source untuk membangun chatbot dan asisten virtual. Rasa dapat dijalankan di server sendiri (self-hosted), sehingga data pelanggan tetap berada di server perusahaan. Dalam pengembangan chatbot customer service, Rasa memiliki tiga fungsi utama, yaitu memahami pesan pengguna, mengelola alur percakapan, dan menjalankan aksi yang dibutuhkan untuk menjawab pertanyaan pengguna.

Komponen Rasa yang digunakan pada setiap tahap ditunjukkan pada tabel berikut.

Tahap	**Komponen Rasa**	**Fungsi**	**Penerapan pada Proyek**
Pemahaman pesan	Pipeline NLU (config.yml) dan NLUCommandAdapter	Mengubah teks pengguna menjadi intent, confidence, dan entity	Teks dikirim ke NLU service eksternal berbasis IndoBERT
	Domain (domain.yml)	Mendefinisikan intent, entity, slot, response, dan action	19 intent dan 4 entity (product, category, order_id, email)
Manajemen dialog	Tracker	Menyimpan riwayat percakapan dan nilai slot	Menyimpan nama produk, nomor pesanan, dan email pelanggan
	Policies	Menentukan aksi berikutnya berdasarkan kondisi percakapan	RulePolicy, MemoizationPolicy, dan TEDPolicy
	Rules, Stories, dan Forms	Mendefinisikan alur percakapan dan pengumpulan data dari pengguna	Form untuk cek produk, status pesanan, dan data pelanggan
Eksekusi aksi	Responses	Jawaban statis chatbot	Sapaan, informasi pengiriman, kebijakan retur
	Custom Actions	Menjalankan kode Python untuk mengambil data dari sistem lain	Mengambil data produk, pesanan, dan pelanggan dari external service
Integrasi	credentials.yml	Menghubungkan chatbot	REST channel untuk UI chat

Tahap	Komponen Rasa	Fungsi	Penerapan pada Proyek
channel		dengan aplikasi pengguna	berbasis web

Arsitektur sistem secara keseluruhan ditunjukkan pada gambar berikut.



Arsitektur sistem chatbot OlshopKu

Sistem terdiri dari empat komponen utama. Rasa Pro (port 5005) bertugas mengelola dialog. NLU service berbasis Flask (port 3000) bertugas mengenali intent dan entity menggunakan model IndoBERT. External service berbasis Node.js dan Express (port 4000) bertugas mengambil data dari database dan menyediakan halaman chat. Database MySQL yang dijalankan melalui MAMP (port 8889) menyimpan data produk, pelanggan, dan pesanan.

b. Explain the differences between RASA's built-in NLU models and external NLU models trained using BERT or DistilBERT. What are the advantages and challenges of using separately trained models? *(b. Jelaskan perbedaan antara model NLU bawaan RASA dan model NLU eksternal yang dilatih menggunakan BERT atau DistilBERT. Apa saja kelebihan dan tantangan dari penggunaan model yang dilatih secara terpisah?)*

NLU bawaan Rasa dan NLU eksternal berbasis BERT memiliki perbedaan pada cara representasi teks, proses training, dan cara penggunaannya. Perbedaan tersebut dirangkum pada tabel berikut.

Aspek	NLU Bawaan Rasa	NLU Eksternal (BERT/DistilBERT)
Representasi teks	Fitur bag-of-words dan n-gram karakter (CountVectorsFeaturizer) dengan classifier DIET yang dilatih dari awal	Embedding kontekstual dari model yang sudah di-pre-train pada korpus besar
Data training	File data/nlu.yml	Dataset dalam format bebas (CSV/XLSX)
Proses training	Dilatih bersamaan dengan model dialog melalui rasa train	Dilatih terpisah menggunakan PyTorch, lalu dipanggil melalui HTTP
Fleksibilitas	Terbatas pada komponen yang disediakan Rasa	Bebas memilih model, preprocessing, metrik evaluasi, dan threshold
Fitur pendukung	Fallback, regex entity, dan sinonim sudah tersedia	Fitur tersebut harus dibuat sendiri di dalam service

Keunggulan model yang dilatih terpisah adalah kemampuannya memahami variasi kalimat dengan lebih baik. Model seperti BERT sudah mempelajari pola bahasa dari korpus yang sangat besar, sehingga kalimat dengan susunan berbeda tetapi bermakna sama tetap dapat dikenali. Selain itu, model NLU dapat dilatih ulang tanpa melatih ulang model dialog, begitu pula sebaliknya. Service NLU yang sama juga dapat digunakan oleh aplikasi lain, misalnya chatbot WhatsApp.

Namun, penggunaan model terpisah juga memiliki beberapa tantangan. Pertama, model BERT membutuhkan sumber daya yang lebih besar. IndoBERT-base memiliki sekitar 110 juta parameter dan setiap pesan membutuhkan satu request HTTP tambahan. Kedua, jika NLU service tidak berjalan, Rasa tidak dapat mengenali pesan pengguna. Ketiga, nama intent pada model harus selalu sama dengan nama intent pada domain.yml. Keempat, preprocessing yang dilakukan saat training dan saat model digunakan harus sama persis agar hasil prediksi tetap konsisten.

Sebagai pembanding, dibuat juga model sederhana menggunakan TF-IDF dan Logistic Regression, yang cara kerjanya mirip dengan CountVectorsFeaturizer. Model ini mencapai akurasi 95,5% pada data uji. Hasil ini menunjukkan bahwa pada dataset berukuran kecil, model sederhana sudah memberikan performa yang cukup baik. Keunggulan model BERT dapat dilihat dari hasil pengujian pada Tabel 2, terutama untuk kalimat yang bervariasi atau tidak baku.

Question-2: Development of external NLU models *(Soal 2: Pengembangan model NLU eksternal)*

a. Present and explain the dataset used to train a customer service NLU model. Examples include intents such as `greet`, `check_customer_data`, `check_order_status`, `ask_product_info`, and `goodbye`. Provide at least two example sentences for each intent. *(a. Paparkan dan jelaskan dataset yang digunakan untuk melatih model NLU layanan pelanggan. Contoh intent antara lain greet, check_customer_data, check_order_status, ask_product_info, dan goodbye. Berikan minimal dua contoh kalimat untuk setiap intent.)*

Dataset yang digunakan disusun dari dua sumber data gratis di internet dan ditambah dengan template kalimat buatan sendiri.

1. Bitext Customer Support LLM Chatbot Training Dataset (lisensi CDLA-Sharing-1.0). Dataset ini berisi 26.872 kalimat customer service berbahasa Inggris dengan 27 intent. Dataset ini digunakan sebagai acuan jenis intent customer service. Sebanyak 84 kalimat dari 9 intent yang relevan, seperti track_order, delivery_options, dan check_payment_methods, diterjemahkan ke Bahasa Indonesia dengan gaya bahasa chat online shop.
2. DummyJSON (lisensi MIT). Dataset ini berisi 194 data produk beserta stok, harga, merek, garansi, dan kebijakan retur, serta 208 data pengguna dan 208 data keranjang belanja. Data ini digunakan untuk mengisi database dan untuk melengkapi kalimat training dengan nama produk, email, dan nomor pesanan yang nyata.
3. Template kalimat buatan sendiri, digunakan untuk intent khas online shop yang tidak tersedia pada Bitext, seperti cek stok, harga, dan kategori produk. Template ini dilengkapi variasi sapaan (“kak”, “min”, “gan”), kata tambahan (“dong”, “ya”), dan salah ketik.

Data training berjumlah 3.040 kalimat yang terbagi rata ke dalam 19 intent, sehingga setiap intent memiliki 160 kalimat. Selain itu, disiapkan data uji terpisah sebanyak 154 kalimat yang ditulis dengan gaya bahasa berbeda dan tidak digunakan saat training. Dataset disimpan dalam file nlu_train.csv, nlu_test_heldout.csv, dan nlu_dataset.xlsx.

Daftar intent beserta contoh kalimatnya ditunjukkan pada tabel berikut.

Intent	**Contoh Kalimat 1**	**Contoh Kalimat 2**
greet	halo kak, selamat sore	assalamualaikum admin
goodbye	oke deh, sampai jumpa	pamit dulu kak
affirm	iya kak	boleh banget
deny	gak jadi kak	batalin aja
thank_you	makasih banyak ya kak	thank you min
bot_challenge	ini bot atau manusia?	kamu AI?
check_stock	kak iPhone X ready ga?	stok samsung galaxy s8 masih ada?
ask_price	berapa harga Lenovo Yoga 920?	rolex datejust lagi diskon gak?
ask_product_info	garansi apple airpods berapa lama?	spesifikasi asus zenbook pro gimana
browse_category	ada parfum apa aja?	toko ini jual apa saja?
check_order_status	pesanan 10023 udah sampe mana?	what is the status of my order?
check_customer_data	tolong tampilkan riwayat transaksi saya	akun saya atas nama siapa?
provide_order_id	nomornya 10045 kak	ORD-10101
provide_email	email saya emily.johnson@x.dummyjson.com	pakai email sophia.brown@x.dummyjson.com
ask_delivery_info	bisa kirim pakai sicepat?	pengiriman ke medan berapa hari?
ask_return_policy	kalau barangnya cacat bisa dikembalikan?	syarat retur apa aja?
contact_admin	minta nomor WA admin dong	saya mau bicara dengan orang langsung
transaction_out_of_scope	bisa bayar pakai QRIS?	tolong pesankan iPhone X 2 biji
out_of_scope	cuaca jakarta hari ini gimana	resep rendang yang enak apa

Intent transaction_out_of_scope dan out_of_scope sengaja ditambahkan agar chatbot dapat mengenali pertanyaan di luar layanannya. Tanpa kedua intent ini, model akan tetap memilih salah satu intent toko meskipun pertanyaan pengguna tidak berhubungan dengan toko.

Dataset ini memiliki beberapa keterbatasan. Sebagian besar kalimat berasal dari terjemahan dan template, bukan dari percakapan pelanggan yang sebenarnya. Data uji juga ditulis oleh penyusun yang sama, sehingga hasil akurasi kemungkinan lebih tinggi dibandingkan saat chatbot digunakan oleh pelanggan sungguhan. Selain itu, data produk DummyJSON berasal dari pasar Amerika, sehingga harga dikonversi ke Rupiah hanya sebagai ilustrasi.

b. Explain the stages of training an intent classification model using BERT or DistilBERT (external model), including dataset preparation, tokenization, training, evaluation, and model saving. *(b. Jelaskan tahapan pelatihan model klasifikasi intent menggunakan BERT atau DistilBERT (model eksternal), mencakup persiapan dataset, tokenization, pelatihan, evaluasi, dan penyimpanan model.)*

Model intent dilatih menggunakan skrip train_intent_model.py. Struktur skrip mengikuti contoh fine-tuning pada modul bab 6 dan 7, yaitu tahap persiapan dataset, training, evaluasi, dan penyimpanan model. Model dasar yang digunakan adalah IndoBERT (indobenchmark/indobert-base-p1), yaitu model BERT yang dilatih khusus untuk Bahasa Indonesia.

1. Persiapan Dataset

Setiap kalimat terlebih dahulu diproses oleh entity extractor. Entity seperti nama produk, kategori, nomor pesanan, dan email diganti dengan placeholder, misalnya `<product>` dan `<order_id>`, seperti teknik modtext pada modul bab 7. Dengan cara ini, model mempelajari pola kalimat dan tidak menghafal nama produk tertentu. Selanjutnya teks diubah menjadi huruf kecil dan kata tidak baku diganti dengan kata baku, misalnya "brp" menjadi "berapa" dan "gak" menjadi "tidak". Label intent kemudian diubah menjadi angka, lalu data dibagi menjadi 85% data training dan 15% data validasi secara stratified.

2. Tokenisasi

Kalimat diubah menjadi token menggunakan tokenizer IndoBERT. Panjang maksimal token diatur sebesar 64 karena pesan chat umumnya pendek. Padding dilakukan secara dinamis pada setiap batch agar proses training lebih efisien.

3. Training

Model dilatih menggunakan AutoModelForSequenceClassification dengan optimizer AdamW, learning rate 3e-5, batch size 16, dan 6 epoch. Proses training dapat berjalan menggunakan GPU Apple Silicon (MPS), CUDA, atau CPU sesuai perangkat yang tersedia.

4. Evaluasi

Pada setiap epoch, model dievaluasi menggunakan data validasi dengan metrik akurasi dan macro-F1. Bobot model dengan nilai macro-F1 tertinggi disimpan sebagai model terbaik. Setelah training selesai, model diuji menggunakan data uji terpisah, kemudian dibuat classification report, confusion matrix, dan daftar kalimat yang salah diprediksi.

5. Penyimpanan Model

Model dan tokenizer disimpan menggunakan fungsi save_pretrained() ke folder models/olshop_intent_model. Nama label intent ikut tersimpan di dalam konfigurasi model, sehingga tidak diperlukan file label encoder terpisah. Informasi training seperti hyperparameter dan hasil evaluasi juga disimpan agar proses training dapat diulang dengan hasil yang sama.

Hasil evaluasi model ditunjukkan pada tabel berikut.

Tabel 2. Hasil evaluasi model intent

Model	Akurasi Validasi	Akurasi Data Uji	Macro-F1 Data Uji
TF-IDF + Logistic Regression	98,7%	95,5%	0,954
IndoBERT (fine-tuned)	0.9846	0.9675	0.9678

Pada model pembanding, kesalahan prediksi paling banyak terjadi pada kalimat yang memang mirip, misalnya "tidak, terima kasih" yang dapat dianggap sebagai deny maupun thank_you,

serta kalimat yang memuat email yang dapat dianggap sebagai provide_email maupun check_customer_data.

c. The external NLU model must produce output that the chatbot system can understand. Define a response format that includes the intent name, confidence score, and entities (if available). Explain how RASA uses this format to process the subsequent dialogue. *(c. Model NLU eksternal harus menghasilkan keluaran yang dapat dipahami oleh sistem chatbot. Tentukan format respons yang memuat nama intent, confidence score, dan entity (jika ada). Jelaskan bagaimana RASA memanfaatkan format tersebut untuk memproses dialog selanjutnya.)*

Format response NLU service mengikuti format hasil parsing Rasa NLU agar dapat langsung dibaca oleh Rasa. Contoh response untuk pesan "kak iPhone X ready ga?" adalah sebagai berikut.

```
{
  "text": "kak iPhone X ready ga?",
  "intent": {"name": "check_stock", "confidence": 0.9731},
  "intent_ranking": [
    {"name": "check_stock", "confidence": 0.9731},
    {"name": "ask_price", "confidence": 0.0112}
  ],
  "entities": [
    {"entity": "product", "value": "iPhone X", "start": 4, "end": 12,
     "confidence_entity": 1.0, "extractor": "GazetteerEntityExtractor"}
  ]
}
```

Keterangan field response:

Field	Fungsi
text	Teks asli yang dikirim pengguna
intent.name	Nama intent hasil prediksi, harus sama dengan intent di domain.yml
intent.confidence	Tingkat keyakinan model (0 sampai 1)
intent_ranking	Daftar intent lain beserta confidence-nya
entities	Daftar entity yang ditemukan, berisi jenis entity, nilai, dan posisinya pada teks

Apabila nilai confidence berada di bawah 0,5, intent diganti menjadi nlu_fallback sehingga chatbot meminta pengguna mengulang pertanyaan. NLU service juga dapat membaca payload tombol seperti `/check_stock` dan perintah `/session_start` yang dikirim oleh Rasa.

Setelah response diterima, Rasa menyimpannya ke dalam tracker sebagai pesan pengguna. Selanjutnya nilai entity digunakan untuk mengisi slot, misalnya entity product mengisi slot product. Policy kemudian menentukan aksi berikutnya berdasarkan intent dan kondisi percakapan. Pada contoh di atas, intent check_stock akan mengaktifkan form produk. Karena slot product sudah terisi, chatbot langsung menjalankan custom action untuk mengambil data stok iPhone X dari database.

Question-3: NLUCommandAdapter Integration *(Soal 3: Integrasi NLUCommandAdapter)*

a. Explain the function of the NLUCommandAdapter within a chatbot architecture that utilizes an external NLU model. Describe the communication flow between RASA and the external NLU service. *(a. Jelaskan fungsi NLUCommandAdapter dalam arsitektur chatbot yang memanfaatkan model NLU eksternal. Uraikan alur komunikasi antara RASA dan layanan NLU eksternal tersebut.)*

NLUCommandAdapter merupakan komponen pada pipeline Rasa Pro yang berfungsi meneruskan hasil klasifikasi intent ke dalam alur dialog. Berdasarkan dokumentasi Rasa Pro, komponen ini mengubah intent menjadi perintah untuk memulai flow tertentu. Pada proyek ini, pengiriman teks ke model NLU eksternal diatur melalui konfigurasi nlu.url pada endpoints.yml. Ketika konfigurasi tersebut diisi, Rasa tidak menjalankan pipeline NLU internal, tetapi mengirim setiap pesan pengguna ke endpoint /model/parse pada NLU service.

Alur komunikasi antara Rasa dan NLU service adalah sebagai berikut.

1. Pengguna mengirim pesan “kak iPhone X ready ga?” melalui halaman chat. Pesan tersebut diteruskan oleh Node.js ke REST channel Rasa.
2. Rasa mengirim request POST ke http://localhost:3000/model/parse dengan body berisi teks pesan dan ID pengirim.
3. NLU service mencari entity pada teks, mengganti entity dengan placeholder, lalu memprediksi intent menggunakan model IndoBERT.
4. NLU service mengembalikan response JSON berisi intent, confidence, dan entity.
5. Rasa menyimpan hasil tersebut ke tracker, mengisi slot, lalu policy menentukan aksi berikutnya.
6. Custom action mengambil data stok dari external service Node.js yang terhubung ke database MySQL.
7. Rasa mengirim jawaban ke halaman chat dan ditampilkan kepada pengguna.

b. Design a REST API-based service using Flask or FastAPI that accepts customer messages and returns intent prediction results from a BERT or DistilBERT model. Describe the endpoints, request format, response format, and error-handling mechanisms. *(b. Rancang layanan berbasis REST API menggunakan Flask atau FastAPI yang menerima pesan pelanggan dan mengembalikan hasil prediksi intent dari model BERT atau DistilBERT. Uraikan endpoint, format request, format response, dan mekanisme penanganan kesalahan (error handling).)*

NLU service dibangun menggunakan Flask dan dijalankan pada port 3000. Service ini menyediakan tiga endpoint seperti pada tabel berikut.

Method	Path	Description	Input Parameters	Return Values
POST	/model/parse	Memprediksi intent dan entity dari pesan pengguna	text, message_id (opsional)	Intent, confidence, intent_ranking, dan entities
GET	/health	Memeriksa status service	–	{ “status”: “ok”, “model_loaded”: true, “n_intents”: 19 }
GET	/model/info	Menampilkan informasi model	–	Daftar intent, threshold, dan informasi training

Request dikirim dalam format JSON melalui request body. Contoh request:

```
{
  "text": "pesanan 10023 udah sampe mana?",
  "message_id": "user-01"
}
```

Jika proses berhasil, service mengembalikan kode status 200 dengan format response seperti yang dijelaskan pada Question 2c. Jika request tidak valid, service mengembalikan kode status 400 beserta pesan kesalahan. Contoh response gagal:

```
{
  "error": "Field 'text' wajib berupa string"
}
```

Mekanisme penanganan error pada service ditunjukkan pada tabel berikut.

Kondisi	Response	Keterangan
Body bukan JSON atau field text tidak ada	400 dengan pesan error	Kesalahan dari sisi client
Method tidak sesuai, misalnya GET pada /model/parse	405 dengan pesan error	Response tetap dalam format JSON
Teks kosong	200 dengan intent nlu_fallback	Rasa tetap menerima format yang valid
Teks lebih dari 500 karakter	Teks dipotong sebelum diproses	Menjaga waktu proses tetap cepat
Confidence di bawah 0,5	200 dengan intent nlu_fallback	Chatbot meminta pengguna mengulang pertanyaan
Terjadi error di server	500 dengan pesan umum	Detail error hanya dicatat di log server
Model belum dilatih	Service tidak dapat dijalankan	Muncul pesan untuk menjalankan training terlebih dahulu

Apabila NLU service tidak dapat dihubungi, Rasa akan menerima intent kosong. Untuk kondisi ini, RulePolicy dikonfigurasi dengan fallback sehingga chatbot tetap memberikan jawaban default kepada pengguna.

c. Design the RASA `config.yml` configuration to utilize the `NLUCommandAdapter` for connecting to an external NLU model. Explain the required configuration parameters and how to ensure the configuration is compatible with the specific version of RASA being used. *(c. Rancang konfigurasi config.yml pada RASA agar memanfaatkan NLUCommandAdapter untuk terhubung dengan model NLU eksternal. Jelaskan parameter konfigurasi yang diperlukan dan cara memastikan konfigurasi tersebut kompatibel dengan versi RASA yang digunakan.)*

Konfigurasi config.yml yang digunakan adalah sebagai berikut.

```
recipe: default.v1
language: id

pipeline:
  - name: NLUCommandAdapter
    url: "http://localhost:3000"
```

```yaml
policies:
  - name: MemoizationPolicy
    max_history: 5
  - name: RulePolicy
    core_fallback_threshold: 0.3
    core_fallback_action_name: "action_default_fallback"
    enable_fallback_prediction: true
  - name: TEDPolicy
    max_history: 5
    epochs: 100
    constrain_similarities: true

assistant_id: olshopku-admin-bot
```

Alamat NLU service juga dituliskan pada endpoints.yml, karena konfigurasi inilah yang digunakan Rasa untuk mengirim pesan ke /model/parse.

```yaml
action_endpoint:
  actions_module: "actions"

nlu:
  url: "http://localhost:3000"
```

Penjelasan parameter yang digunakan:

Parameter	**Fungsi**
recipe	Resep training standar Rasa (default.v1)
language	Kode bahasa chatbot (id untuk Bahasa Indonesia)
pipeline	Hanya berisi NLUCommandAdapter karena proses NLU dilakukan oleh service eksternal
MemoizationPolicy	Mengingat alur percakapan yang sama persis dengan stories
RulePolicy	Menjalankan rules dan forms, serta menangani fallback dialog
TEDPolicy	Mempelajari pola percakapan dari stories agar dapat menangani alur yang belum pernah dilihat
assistant_id	Identitas unik model chatbot
nlu.url (endpoints.yml)	Alamat NLU service yang dipanggil Rasa
actions_module (endpoints.yml)	Lokasi file custom action

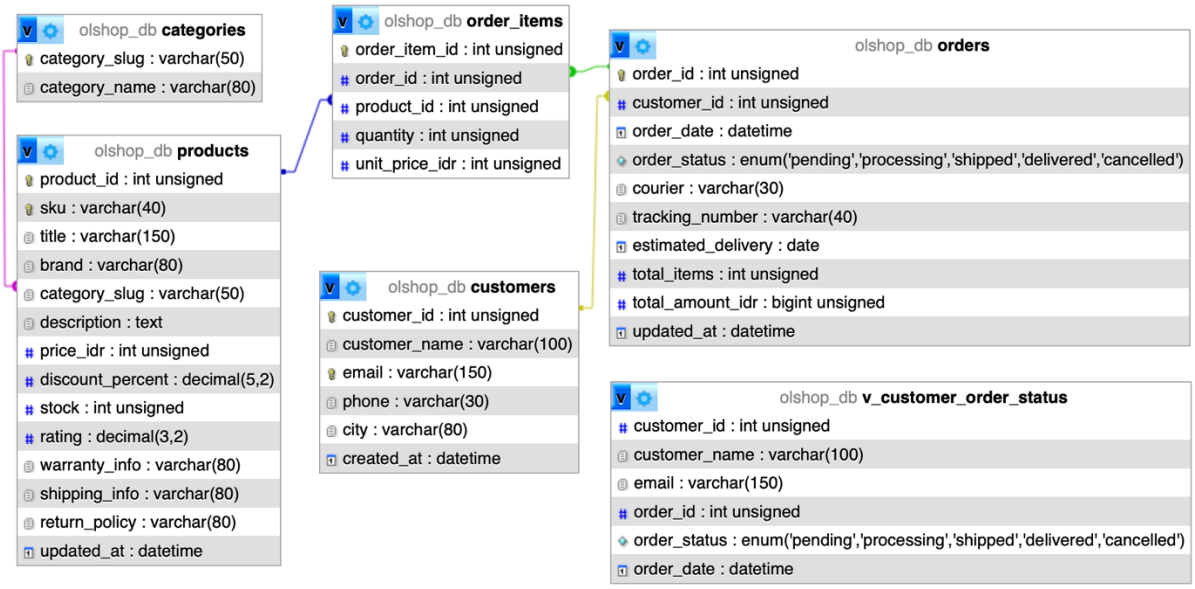
Untuk memastikan konfigurasi sesuai dengan versi Rasa yang digunakan, dilakukan beberapa langkah berikut. Pertama, versi Rasa Pro ditentukan saat instalasi dan dicek menggunakan perintah rasa –version. Versi yang digunakan pada proyek ini adalah Rasa Pro 3.21.0. Nilai version: “3.1” pada file YAML menunjukkan versi format data, bukan versi Rasa. Kedua, perintah rasa data validate dijalankan sebelum training untuk memastikan file konfigurasi dan data training valid. Ketiga, file data/nlu.yml dan data/flows.yml tidak digunakan, sehingga rasa train hanya melatih model dialog sesuai ketentuan dokumentasi Rasa untuk NLU eksternal. Jika versi Rasa yang digunakan tidak mendukung parameter url pada NLUCommandAdapter atau actions_module, kedua parameter tersebut dapat dihapus atau diganti dengan action server

terpisah (rasa run actions). Selain itu, environment Python untuk Rasa dan untuk PyTorch dipisahkan agar versi library keduanya tidak saling bentrok.

Question-4: Integration of external customer databases *(Soal 4: Integrasi basis data pelanggan eksternal)*

a. Design a customer database schema that includes, at a minimum, `customer_id`, `customer_name`, `email`, and `order_status`. Explain why customer data should be stored in an external database rather than being hardcoded directly into the chatbot's code. *(a. Rancang skema basis data pelanggan yang setidaknya memuat customer_id, customer_name, email, dan order_status. Jelaskan mengapa data pelanggan sebaiknya disimpan dalam basis data eksternal dan tidak ditulis langsung (hardcoded) di dalam kode chatbot.)*

Database olshop_db dirancang menggunakan MySQL yang dijalankan melalui MAMP. Database terdiri dari lima tabel, yaitu customers, orders, order_items, products, dan categories. Relasi antar tabel ditunjukkan pada ERD berikut.



ERD database olshop_db

Kolom customer_id, customer_name, dan email disimpan pada tabel customers, sedangkan order_status disimpan pada tabel orders. Pemisahan ini dilakukan karena satu pelanggan dapat memiliki banyak pesanan dengan status yang berbeda-beda. Jika order_status disimpan pada tabel customers, status pesanan lama akan tertimpa oleh status pesanan baru. Untuk menampilkan keempat kolom tersebut sekaligus, dibuat view berikut.

```
CREATE VIEW v_customer_order_status AS
SELECT c.customer_id, c.customer_name, c.email,
       o.order_id, o.order_status, o.order_date
FROM customers c JOIN orders o ON o.customer_id = c.customer_id;
```

Kolom email dibuat unik karena digunakan sebagai identitas pelanggan. Kolom order_status menggunakan tipe ENUM dengan nilai pending, processing, shipped, delivered, dan cancelled. Status ketersediaan stok tidak disimpan sebagai kolom terpisah, tetapi dihitung dari jumlah stok agar tidak terjadi perbedaan data. Data sensitif dari DummyJSON seperti password dan nomor rekening juga tidak dimasukkan ke dalam database.

Data pelanggan disimpan pada database eksternal karena beberapa alasan berikut.

1. Data stok dan status pesanan terus berubah. Jika data ditulis langsung di dalam kode, setiap perubahan mengharuskan kode diedit dan chatbot dijalankan ulang. Dengan database terpisah, chatbot selalu membaca data terbaru.
2. Database menjadi sumber data tunggal. Website toko, aplikasi gudang, dan chatbot membaca data yang sama sehingga tidak terjadi perbedaan informasi.
3. Keamanan data lebih terjaga. Data pribadi pelanggan tidak ikut tersimpan di dalam kode program maupun model, dan akses ke data hanya dapat dilakukan melalui API yang memiliki validasi.
4. Tanggung jawab setiap komponen menjadi jelas. Chatbot hanya mengatur percakapan, sedangkan external service mengatur data, sehingga keduanya dapat dikembangkan dan diuji secara terpisah.

b. Explain the function of custom actions in Rasa and how this component is used to retrieve customer data from an external server. Create pseudocode for the `action_check_customer_data` action, which accepts a customer's identity, sends a request to a database server, and returns the result to the chatbot. *(b. Jelaskan fungsi custom action pada Rasa dan bagaimana komponen ini digunakan untuk mengambil data pelanggan dari server eksternal. Buatlah pseudocode untuk action_check_customer_data, yaitu action yang menerima identitas pelanggan, mengirim permintaan ke server basis data, lalu mengembalikan hasilnya ke chatbot.)*

Custom action merupakan kode Python yang dijalankan Rasa ketika chatbot perlu melakukan proses yang tidak dapat ditangani oleh response statis, misalnya mengambil data dari database. Custom action dibuat dengan menurunkan class Action dari rasa_sdk. Di dalam custom action, nilai slot dapat dibaca melalui tracker, jawaban dikirim melalui dispatcher, dan nilai slot dapat diperbarui menggunakan event SlotSet.

Pada proyek ini, custom action tidak mengakses database MySQL secara langsung, tetapi mengirim request HTTP ke external service Node.js. Dengan cara ini, informasi koneksi database tidak perlu disimpan di dalam kode chatbot, dan proses validasi data cukup dilakukan di satu tempat. Pola ini sama dengan contoh minta_transkrip_action pada modul bab 7.

Pseudocode action_check_customer_data adalah sebagai berikut.

```
ACTION action_check_customer_data(dispatcher, tracker):
    email = tracker.get_slot("customer_email")

    IF email kosong:
        tampilkan pertanyaan "email yang terdaftar apa?"
        RETURN []
```

```
kirim request GET ke /api/customers/lookup?email=<email>
dengan batas waktu 8 detik

IF request gagal atau server error:
    tampilkan pesan "sistem sedang gangguan"
    RETURN []

IF status = 404:
    tampilkan pesan "email belum terdaftar"
    RETURN [SlotSet("customer_email", null)]

ambil data pelanggan dan riwayat pesanan dari response
tampilkan nama, email, nomor HP (disamarkan), kota,
          tanggal bergabung, total pesanan, dan riwayat pesanan
RETURN []
```

Endpoint yang digunakan oleh custom action disediakan oleh external service Node.js seperti pada tabel berikut.

Method	Path	Description	Input Parameters	Return Values
GET	/api/products/search	Mencari produk berdasarkan nama	q, limit	Daftar produk beserta harga dan stok
GET	/api/products	Menampilkan produk per kategori	category, limit	Daftar produk pada kategori tersebut
GET	/api/categories	Menampilkan daftar kategori	–	Daftar kategori dan jumlah produk
GET	/api/customers/lookup	Menampilkan data pelanggan	email	Profil pelanggan dan riwayat pesanan
GET	/api/orders/{orderId}	Menampilkan detail pesanan	orderId, email	Status, kurir, resi, dan isi pesanan

Pada sisi server, endpoint /api/customers/lookup terlebih dahulu memeriksa format email. Jika format tidak valid, server mengembalikan kode 400. Selanjutnya server mencari data pelanggan pada tabel customers. Jika data tidak ditemukan, server mengembalikan kode 404. Jika data ditemukan, server mengambil ringkasan dan riwayat pesanan dari tabel orders, kemudian menyamarkan nomor HP sebelum mengirimkan response. Contoh response berhasil adalah sebagai berikut.

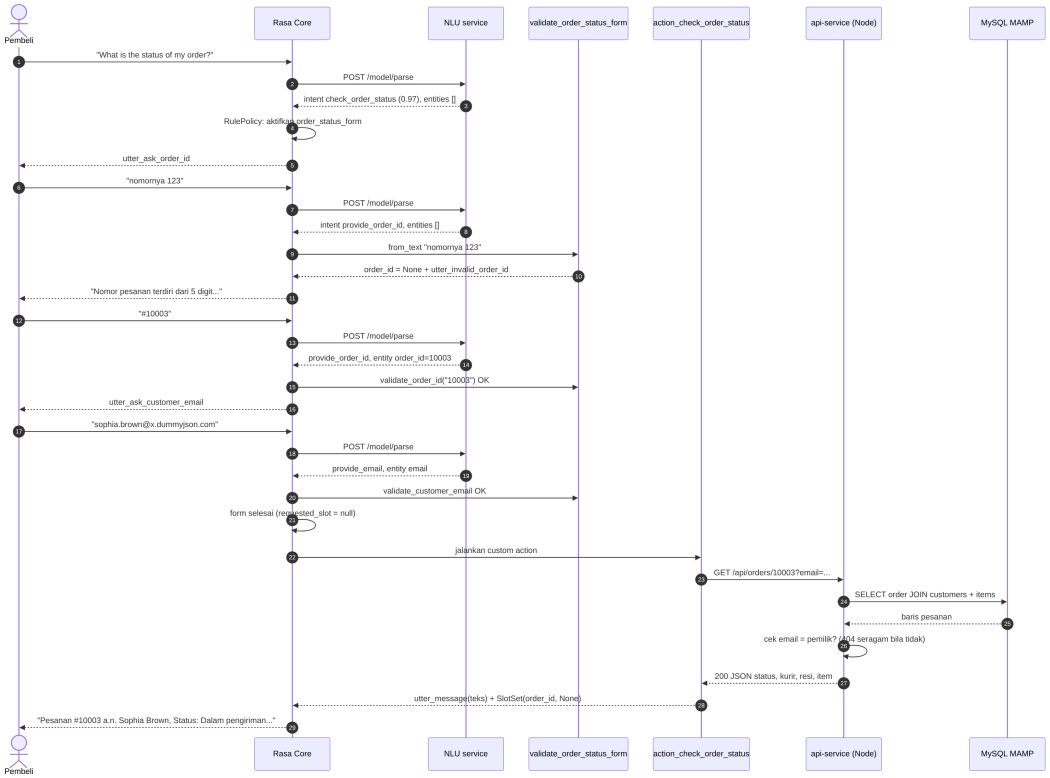

```
{
  "success": true,
  "data": {
    "customer": {
      "customer_id": 1,
      "customer_name": "Emily Johnson",
      "email": "emily.johnson@x.dummyjson.com",
      "phone_masked": "********3024",
      "city": "Phoenix"
    },
    "summary": { "total_orders": 1 },
```

```
    "recent_orders": [ { "order_id": 10001, "status_label": "Sudah diterima"
} ]
  }
}
```

Seluruh query menggunakan placeholder sehingga aman dari SQL injection, dan detail error database tidak dikirimkan ke client.

c. Design the dialogue flow for when a customer asks (example), “What is the status of my order?” Explain how the chatbot obtains the customer’s identity, validates the input, requests data from an external server, and conveys the results to the user. *(c. Rancang alur dialog ketika pelanggan bertanya, misalnya, "Bagaimana status pesanan saya?" Jelaskan bagaimana chatbot memperoleh identitas pelanggan, memvalidasi masukan, meminta data dari server eksternal, dan menyampaikan hasilnya kepada pengguna.)*

Alur dialog ketika pelanggan menanyakan status pesanan ditunjukkan pada sequence diagram berikut.



Sequence diagram alur cek status pesanan

Ketika pelanggan mengirim pesan “What is the status of my order?”, NLU service mengenali pesan tersebut sebagai intent check_order_status. Rasa kemudian mengaktifkan order_status_form yang membutuhkan dua data, yaitu nomor pesanan (order_id) dan email pelanggan (customer_email).

Pada tahap memperoleh identitas, chatbot terlebih dahulu memeriksa apakah nomor pesanan sudah disebutkan pada pesan pertama. Jika pelanggan menulis “status pesanan 10003”, nomor

pesanan langsung dikenali oleh NLU service dan disimpan ke slot. Jika belum ada, chatbot menanyakan nomor pesanan, lalu menanyakan email yang terdaftar. Email yang sudah diverifikasi sebelumnya dalam sesi yang sama tidak ditanyakan kembali.

Setiap input divalidasi oleh validate_order_status_form. Nomor pesanan harus terdiri dari 5 digit, dan pelanggan boleh menuliskannya dalam beberapa bentuk seperti “#10003” atau “ORD-10003”. Email diperiksa formatnya. Jika input tidak valid, chatbot menjelaskan format yang benar dan menanyakan kembali data tersebut.

Setelah kedua data valid, action_check_order_status mengirim request GET /api/orders/{order_id}?email={email} ke external service. Server mencari pesanan di database dan memeriksa apakah email yang diberikan sesuai dengan pemilik pesanan. Jika pesanan tidak ditemukan atau email tidak sesuai, server mengembalikan pesan yang sama. Hal ini dilakukan agar orang lain tidak dapat menebak nomor pesanan milik pelanggan lain.

Jika data ditemukan, chatbot menampilkan status pesanan dalam bahasa yang mudah dipahami, seperti “Dalam pengiriman”, beserta tanggal pesanan, kurir, nomor resi, estimasi tiba, daftar produk, dan total belanja. Setelah itu, slot nomor pesanan dikosongkan agar pertanyaan berikutnya dapat menggunakan nomor pesanan yang berbeda.

Contoh percakapan:

```
User : What is the status of my order?
Bot  : Boleh minta nomor pesanannya, kak? (5 digit, contoh: 10023)
User : nomornya 123
Bot  : Hmm, "nomornya 123" bukan format nomor pesanan yang benar, kak.
       Nomor pesanan terdiri dari 5 digit, contoh 10023.
User : #10003
Bot  : Untuk verifikasi, email yang terdaftar di akun kakak apa?
User : sophia.brown@x.dummyjson.com
Bot  : Pesanan #10003 a.n. Sophia Brown
       Status: Dalam pengiriman
       Kurir: ... Resi: ...
       Estimasi tiba: ...
       Total: Rp...
Bot  : Ada lagi yang bisa Mimin bantu, kak?
```

Chatbot juga menangani beberapa skenario gagal. Apabila pelanggan membatalkan proses, misalnya dengan mengetik “batal”, form dihentikan dan chatbot mengonfirmasi bahwa permintaan dibatalkan. Apabila external service tidak dapat dihubungi, chatbot menampilkan pesan bahwa sistem sedang mengalami gangguan tanpa menghentikan percakapan. Apabila confidence hasil NLU rendah, chatbot meminta pelanggan mengulang pertanyaan dengan kalimat lain.

Perlu diperhatikan bahwa penggunaan email sebagai identitas hanya sesuai untuk keperluan demo. Pada sistem yang sebenarnya, identitas pelanggan sebaiknya diperoleh dari akun yang sudah login atau diverifikasi menggunakan kode OTP. Dengan demikian, alur dialog ini sudah mencakup proses memperoleh identitas, validasi input, permintaan data ke server, dan penyampaian hasil kepada pelanggan.

Question-4: Create a short video *(Soal 4: Membuat video singkat)*

a. Create a short video (maximum 20 minutes) explaining how you built the chatbot, from start to finish, including a demo. Upload it to YouTube. *(a. Buatlah video singkat (maksimal 20 menit) yang menjelaskan cara Anda membangun chatbot dari awal hingga akhir, termasuk demo. Unggah video tersebut ke YouTube.)*

Video menjelaskan proses pembuatan chatbot mulai dari penyusunan dataset, pembuatan database di MAMP, training model IndoBERT, pembuatan NLU service dan external service, konfigurasi Rasa, hingga demo percakapan menggunakan halaman chat.

No	Description	Link
a	Video Tutorial	_____
b	Source Code	_____